

CSE327

Software Engineering

AKM IQTIDAR NEWAZ [IQN]

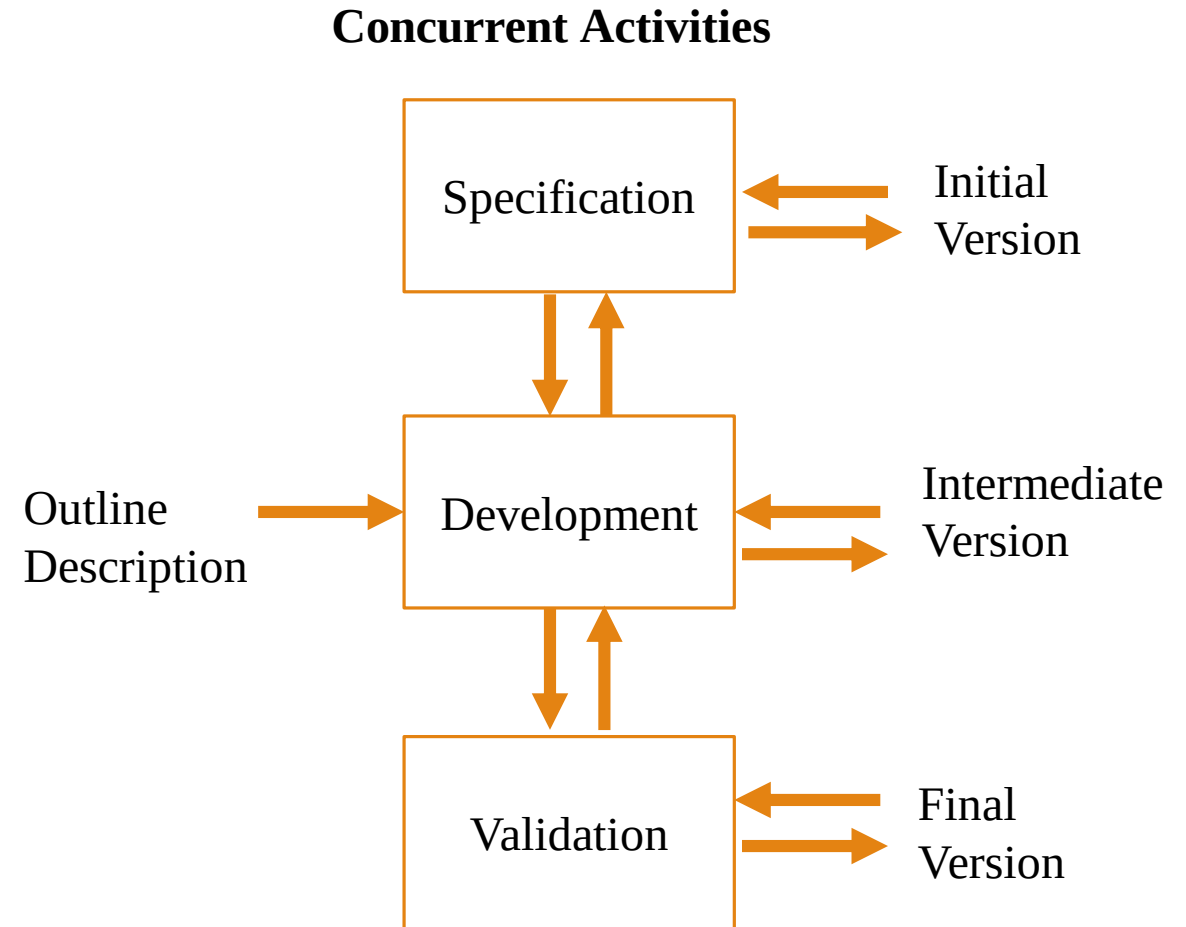
LECTURE -3



Evolutionary Model / Incremental Model

What is Evolutionary model?

- Evolutionary model is also referred to as the successive versions model and sometimes as the incremental model. In Evolutionary model, the software requirement is first broken down into several modules (or functional units) that can be incrementally constructed and delivered.



Evolutionary Model / Incremental Model (Continued)

- ❑ The developers first develop the core modules of the system.
- ❑ The core modules are those that do not need services from the other modules.
- ❑ The initial product skeleton is refined into increasing levels of capability by adding new functionalities in successive versions.
- ❑ Each evolutionary model may be developed using an iterative waterfall model of development.
- ❑ The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation.
- ❑ Evolutionary model is used when the customer prefers to receive the product in increments so that he can start using the different features as and when they are developed rather than waiting all the time for the full product to be developed and delivered.

Evolutionary Model / Incremental Model (Continued)

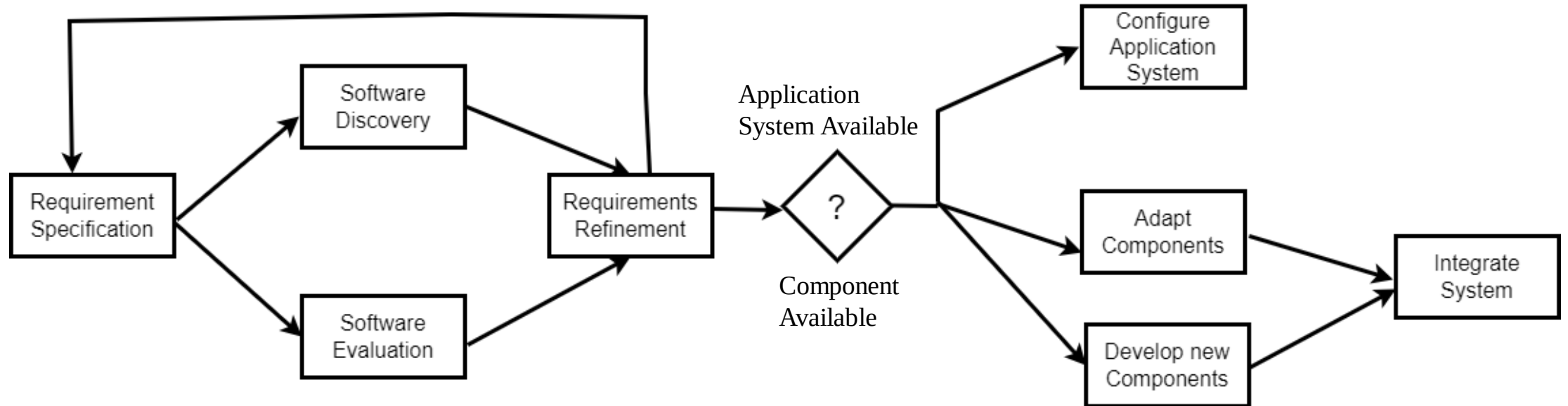
□ Advantage:

- This is mainly used in large software.
- Communicate with clients frequently and always gets feedback.
- User gets a chance to experiment with a partially developed software much before the complete version of the system is released.
- The core modules get tested thoroughly, thereby reducing the chances of errors in the core modules of the final products.

□ Disadvantage:

- Difficult to divide the problem into several versions that would be acceptable to the customer and which can be incrementally implemented and delivered.

Re-use Oriented Software Engineering



Re-use Oriented Software Engineering

❑ Major processing stages:

- Requirement specification
- Software discovery and evaluation
- Requirement refinement
- Application system configuration
- Component adaptation and integration

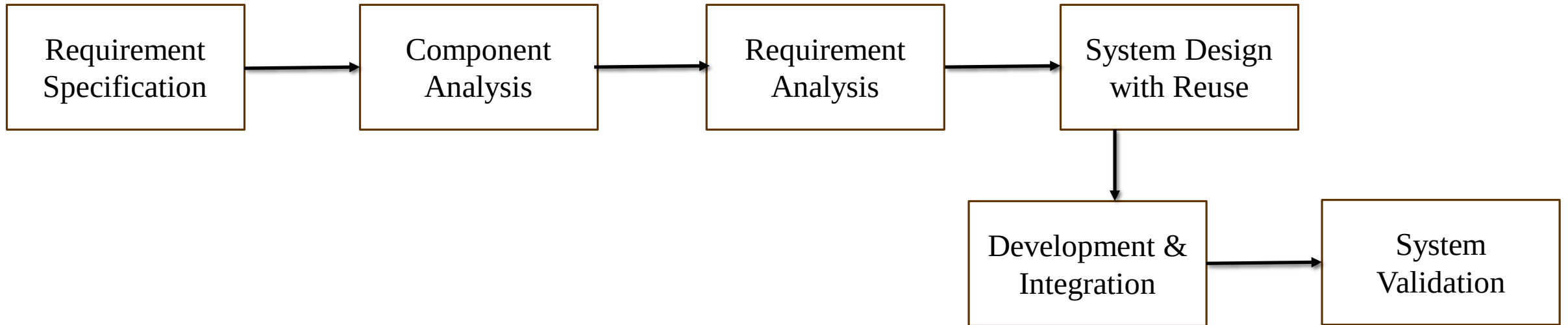
❑ Advantage:

- Reduced costs and risks as less software is developed from scratch
- Faster delivery and deployment of system

❑ Disadvantage:

- But requirements compromises are inevitable so system may not meet real needs of users
- Loss of control over evolution of reused system elements

Component-Based Software Engineering



Component-Based Software Engineering

- While the initial requirements specification stage and the validation stage are comparable with other processes, the intermediate stages in a reuse-oriented process are different. These stages are:
- Component analysis Given the requirements specification; a search is made for components to implement that specification. Usually, there is no exact match, and the components that may be used only provide some of the functionality required.

Component-Based Software Engineering

- ❑ Requirements modification During this stage, the requirements are analyzed using information about the components that have been discovered. They are then modified to reflect the available components. Where modifications are impossible, the component analysis activity may be re-entered to search for alternative solutions.
- ❑ System design with reuse During this phase, the framework of the system is designed, or an existing framework is reused. The designers take into account the components that are reused and organize the framework to cater to this. Some new software may have to be designed if reusable components are not available.

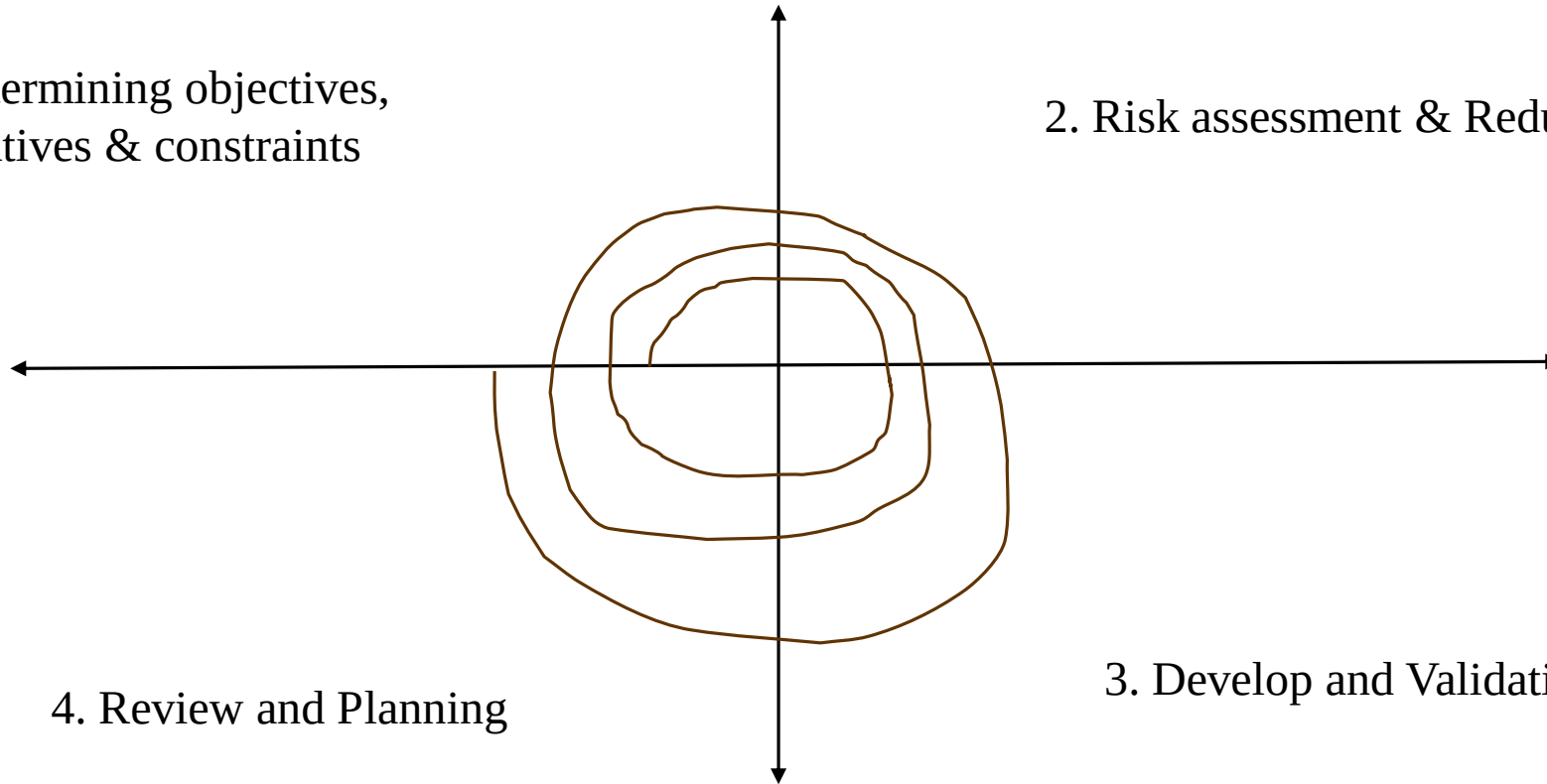
Component-Based Software Engineering

- ❑ Development and integration Software that cannot be externally procured is developed, and the components and COTS systems are integrated to create the new system. System integration, in this model, may be part of the development process rather than a separate activity.
- ❑ Advantage:
 - Reduce the amount of software to be developed and so' reducing cost and risks.
 - It usually also leads to faster delivery of the software.
- ❑ Disadvantage:
 - However, requirements compromises are inevitable, and this may lead to a system that does not meet the real needs of users.
 - Furthermore, some control over the system evolution is lost as new versions of the reusable components are not under the control of the organization using them.

Spiral Model

1. Determining objectives,
Alternatives & constraints

2. Risk assessment & Reduction



3. Develop and Validation

4. Review and Planning

Spiral Model (Continued)

- ❑ The spiral model is a software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the linear sequential model.
- ❑ The spiral model is also known as meta model since it encompasses all other life cycle models. There are four phases:
 - ❑ First Quadrant (Objective Setting)
 - Identify the objectives of the phase.
 - Examine the risks associated with these objectives.

Spiral Model (Continued)

- ❑ Second Quadrant (Risk Assessment and Reduction)
 - A detailed analysis is carried out for each identified project risk.
 - Steps are taken to reduce the risks.

- ❑ Third Quadrant (Development and Validation)
 - Develop and validate the next level of the product after resolving the identified risks.

- ❑ Fourth Quadrant (Review and Planning)
 - Review the results achieved so far with the customer and plan the next iteration.
 - Progressively more complete version of the software gets built with each iteration around the spiral.

Spiral Model (Continued)

□ Advantage:

- Suitable for large projects: It is suggested to use the spiral model for large and complex projects.
- Risk handling capability: Project which is complex and large are prone to unknown risk and error.
- The spiral model is suitable under these high-risk environments. New requirement: Based on the feedback, the requirements are redefined.

□ Disadvantage:

- It is one of the most complex SDLC models.
- Documentation is more as it has many loop iteration.
- It is not suitable for small projects.
- Management is more complex here.